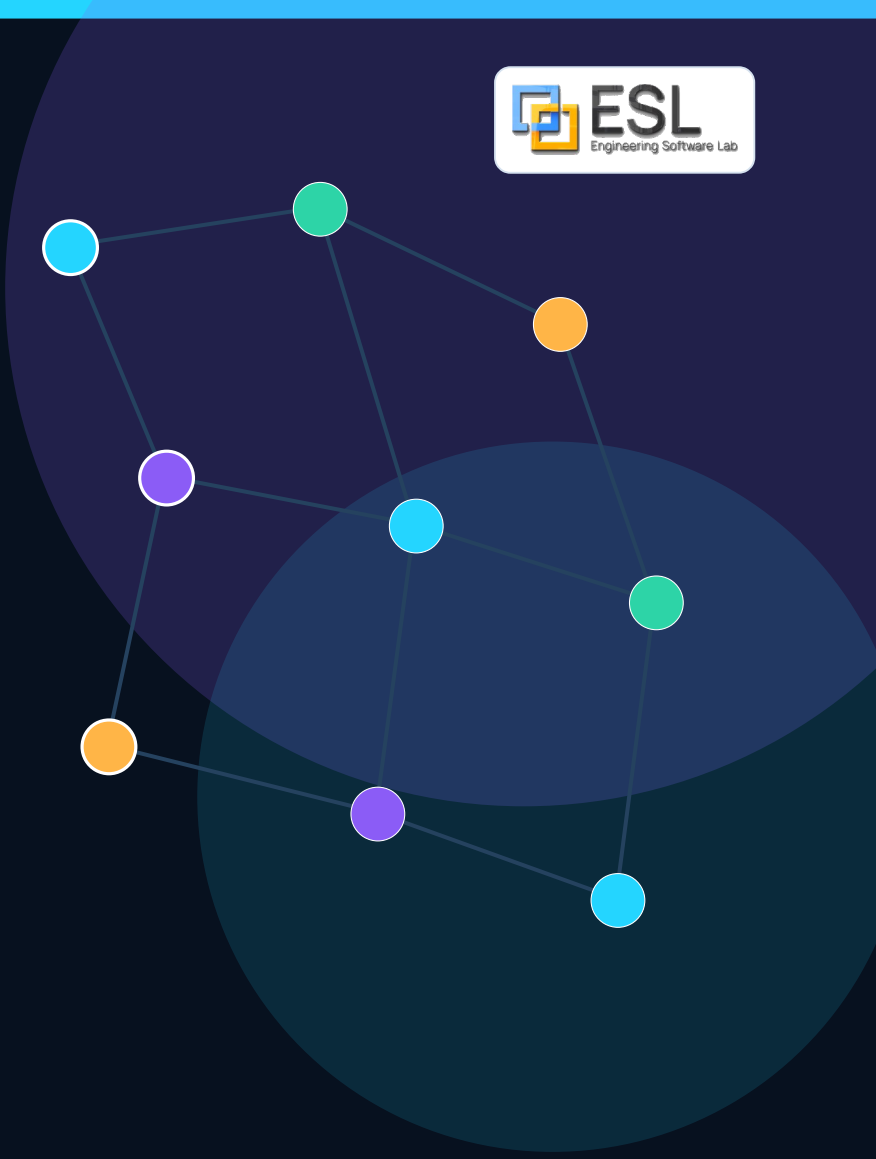
PRODUCTION CONTEXT

Mysticeti Consensus Digital Twin

Amp + Wolfram Mathematica + Lean 4 + Rust

An executable + formally bounded microscope for the quorum and direct-decision core behind production Sui Layer-1 consensus.



Scientific boundary Bounded Rust research twin · not production equivalence · not a production benchmark reproduction

Why Mysticeti matters

A research result with a real deployment path — and an unusually sharp latency target.

Oct 2023

Preprint

"Reaching the Latency Limits with Uncertified DAGs"

25 Jul 2024

Sui Mainnet

Validators switched to Mysticeti-C consensus.

NDSS 2025

Peer-reviewed

Mysticeti appears in the NDSS 2025 program.

Goal

3 message rounds

Direct commit path targets three message rounds.

SOURCE TRAIL arxiv.org/abs/2310.14821 · ndss-symposium.org/ndss-paper/mysticeti · docs.sui.io/.../consensus

Production relevance raises the assurance bar: every claim needs a named evidence lane.

One protocol. Five different kinds of claim.

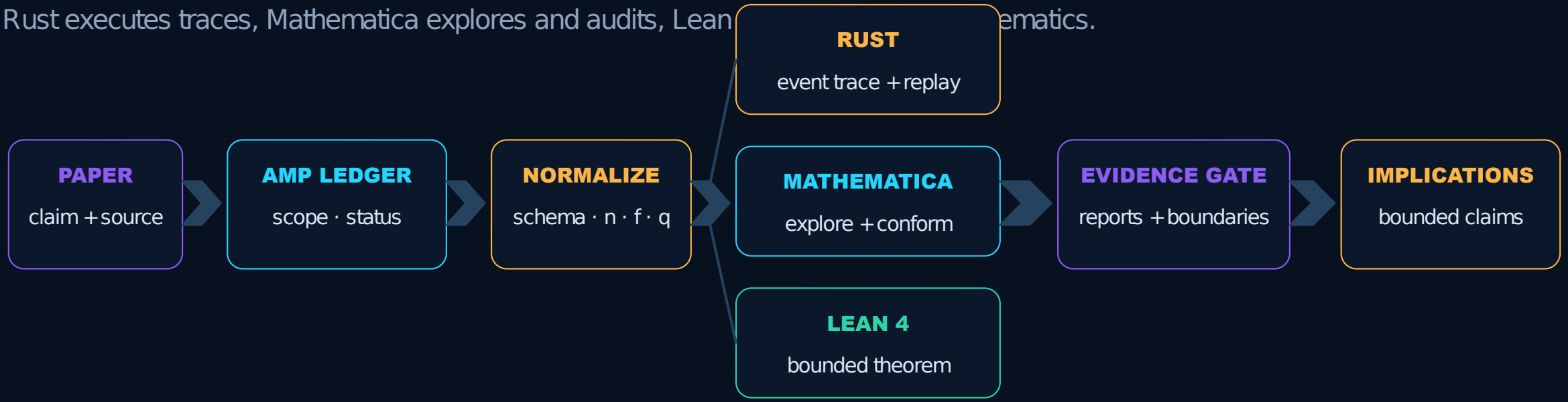
The challenge is not generating output — it is preventing evidence from silently changing meaning.

PAPER CLAIM	**What the authors state**	citation + normalized claim	TRACEABLE
RUST DIGITAL TWIN	**What bounded traces execute**	event-driven replay + audit	TRACEABLE
FORMAL PROOF	**What follows from assumptions**	Lean kernel-checked theorem	TRACEABLE
REPORTED DATA	**What Table I reports**	local CSV transcription	TRACEABLE
PRODUCTION SUI	**What deployed Sui does**	public sources; parity not claimed	OUT OF SCOPE

No upward inference without a bridge.

Amp coordinates three lanes — one evidence gate

Rust executes traces, Mathematica explores and audits, Lean ematics.



EVIDENCE GATE **Every output carries: source → assumption → method → result → limitation.**

Evidence hierarchy — strong does not mean broad

Confidence is attached to the exact statement, not inherited by neighboring protocol claims.

FORMALLY PROVED — Lean checks the theorem from explicit finite-set assumptions.

RUST EXECUTABLE — Bounded stake-weighted event traces, strict replay and audit.

MATHEMATICA — Independent conformance, fixtures and visualization.

CSV TRANSCRIPTION — Locally stored paper values — not independent reproduction.

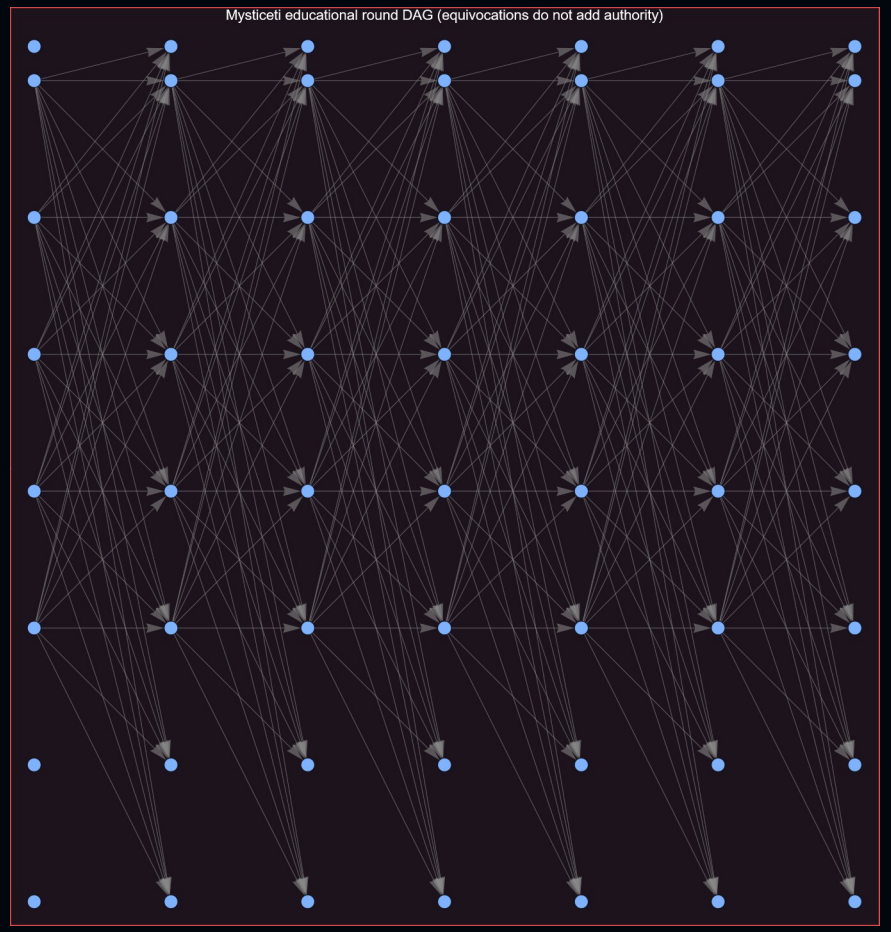
OUT OF SCOPE — Production parity · full protocol proof · liveness · performance.

STRONG + NARROW

NO CLAIM

Read the DAG as evidence — not as decoration

Rounds run left-to-right in the source graphic; validator lanes and authority identity govern every count.



ROUNDS — Causal depth, not wall-clock time

AUTHORITY — One validator = one voting identity

EQUIVOCATION — More blocks ≠ more voting power

PARENTS — Distinct-authority references

The generated fixture explicitly checks parent references and distinct parent authorities.

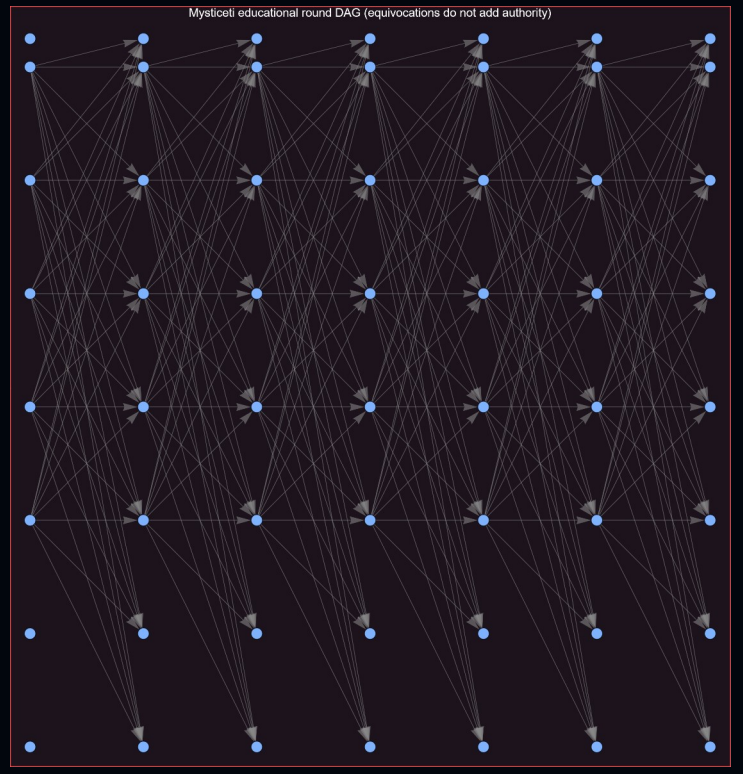
A certificate is not a commit

The model separates evidence at r, r+1, and r+2 so "I saw support" cannot become accidental finality.

DIRECT SKIP **At r + 1, q distinct non-support authorities can justify a conservative direct skip — no r + 2 certificate quorum required.**

Consensus Safety Microscope

A developer changes assumptions, predicts the outcome, then receives inspectable evidence — not a magic score.



fault bound f

crashed validators

round count

equivocation

random seed

RETURNED EVIDENCE

Decision	**Commit / Skip / Undecided**
Support	authority IDs at r + 1
Certificates	block + authority IDs at r + 2
Threshold	explicit q = 2f + 1
Invariants	parents valid · authorities distinct

PREDICT → VARY → INSPECT

The executable twin is small enough to inspect — complete enough to challenge

Every transition is deterministic; every recorded claim can be replayed and independently audited.

COMMITTEE
validated, stake-weighted

CANONICAL DAG
transactional insertion

LOCAL VIEWS
one per authority

EVENT QUEUE
deterministic ordering

DECISION ENGINE
direct commit / skip

CAMPAIGN ENGINE
dedicated Rayon pool

STRICT REPLAY
schema + tamper checks

INVARIANT AUDITOR
independent evidence checks

TRACE CONTRACT `inputs → receiver-specific events → local DAG transitions → decisions → invariant evidence → strict replay`

RESEARCH TWIN ≠ PRODUCTION SUI

Faults matter because delivery changes what each authority can know

The simulator models causal consequences, not just labels attached after a global DAG is built.

SEND block created → **DELIVER / DROP** per receiver → **LOCAL VIEW** receiver DAG → **DEADLINE** round closes → **PARENTS** selected locally

CRASH stops future sends

LOSS removes receiver-specific delivery

EQUIVOCATION creates competing blocks; stake still deduplicated

EVIDENCE LIMIT Global campaign aggregates deterministic research scenarios; it is not a calibrated WAN model or Sui production benchmark.

The release evidence is executable, not decorative

Actual Windows + Ubuntu WSL reports: tests, clippy, trace replay, deterministic campaigns, and Criterion evidence.

WSL / Linux reproducibility evidence

```
$ cat exports/wsl_rust_build_report.txt
Ubuntu 24.04.3 LTS  |  WSL2  |  24 vCPUs
Rust test result: 26 passed; 0 failed; 0 ignored
cargo clippy --all-targets --all-features -- -D warnings   PASS

TRACE  Windows == Ubuntu WSL
6272fa854de66bc42512f38d095d7ccf6f75bb85f581dc4acaabf9fbe8ede71d
BYTE_IDENTICAL_WINDOWS_WSL_TRACE=true

CAMPAIGN  Win jobs=1 == Win jobs=8 == WSL jobs=1 == WSL jobs=8
20 seeds / 80 rows
5fdc665f5c35cee5c63143860789a9a2a4db831ec65c9e814610d1f6b1764a6a
BYTE_IDENTICAL_WINDOWS_WSL_JOBS1_JOBS8=true

$ cat exports/wsl_benchmark_report.txt
Criterion 32 authorities / 50 slots | 10 samples
time: [2.9226 s, 2.9977 s estimate, 3.0633 s]
Development-machine WSL evidence; not bare-metal or production.
```

26 TESTS · 0 FAILED / IGNORED

PARALLEL CAMPAIGN TESTS

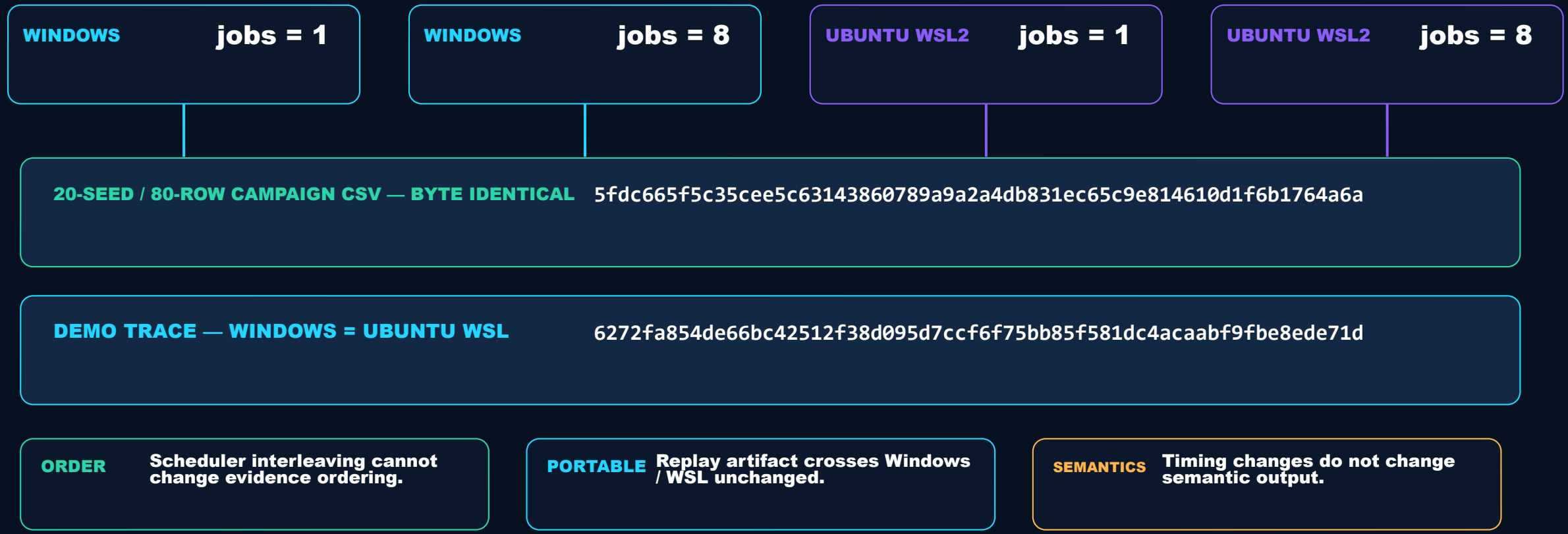
CLIPPY: WINDOWS + WSL

WINDOWS [3.5361, 3.5774] s

WSL EST. 2.9977 s

Same evidence across OS and scheduler

Independent scenario concurrency changes timing and interleaving — never canonical evidence ordering.



ORDER Scheduler interleaving cannot change evidence ordering.

PORTABLE Replay artifact crosses Windows / WSL unchanged.

SEMANTICS Timing changes do not change semantic output.

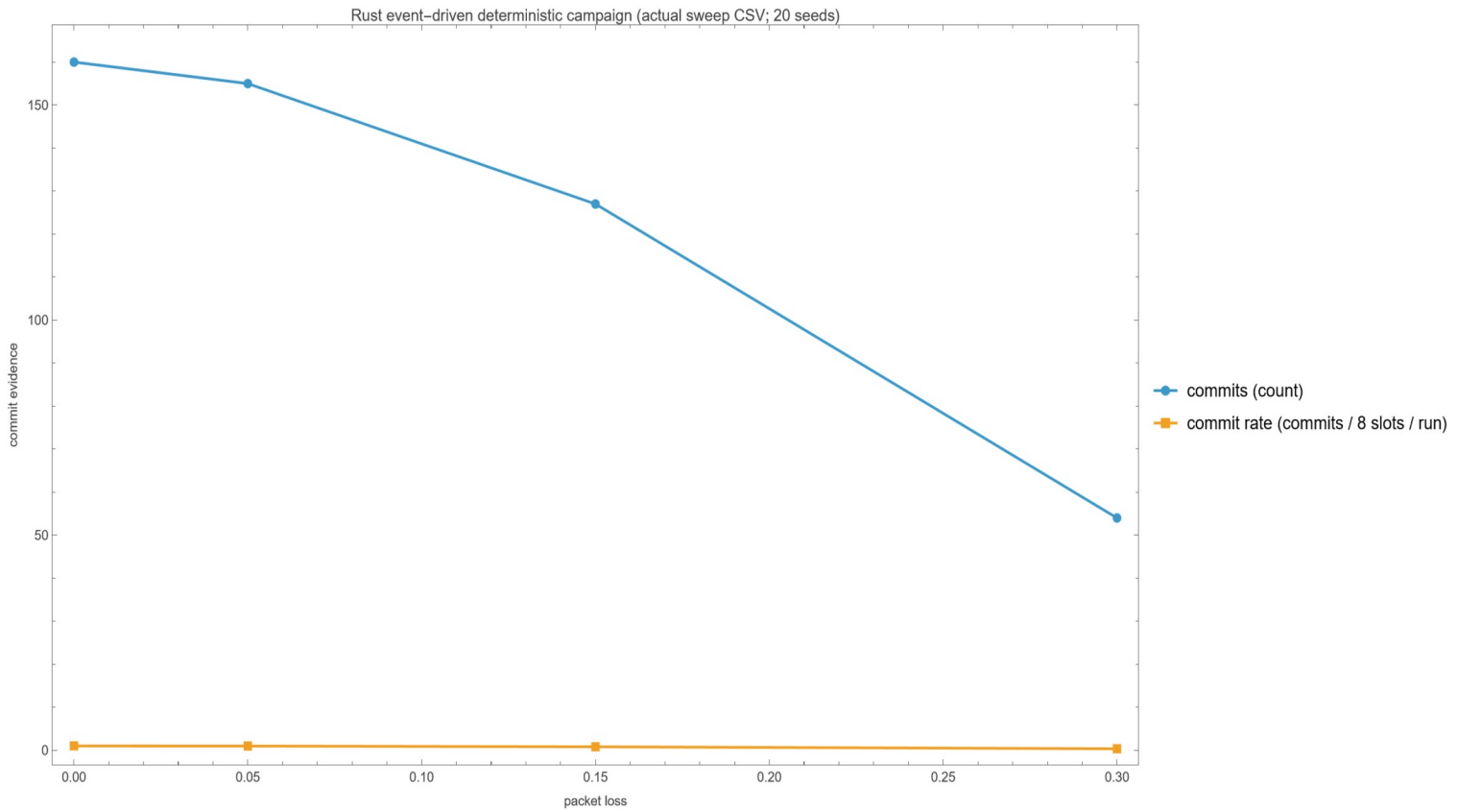
BOUNDARY Dedicated Rayon pool across independent scenarios · each scenario deterministic · canonical sort before CSV serialization · no global pool mutation.

WSL CRITERION 32 authorities / 50 slots · [2.9226 s, 2.9977 s estimate, 3.0633 s] · 10 samples · Ubuntu WSL2 · 24 vCPUs

NON-CLAIMS Development-machine WSL evidence; not bare-metal/production; no speedup comparison against Windows; no Ethereum builder flow or Sui production implementation.

Loss reduces progress in the bounded Rust campaign

20 seeds × 4 loss levels = 80 rows; dedicated Rayon pool across independent deterministic scenarios.



LOSS 0.00

8.0 average commits

LOSS 0.30

2.7 average commits

DETERMINISM BOUNDARY

Canonical sort before CSV; no global pool mutation.

A reusable Wolfram architecture — not a notebook monolith

Deterministic package APIs feed notebook exploration, fixtures, plots, CSV evidence, and release reports.

GenerateMysticetiDAG
seeded round DAG

SupportedProposal
deterministic BFS

CertificateEvidence
q supporters / authority

DirectSlotDecision
commit · skip · undecided

RunValidationSuite
12 named checks

GENERATED EVIDENCE

```
dag.png
safety_envelope.png
fault_frontier.png
validation_results.json
```

SupportedProposal[dag_, block_, slot_] :=

```
breadth-first ancestors  →  sort {round, validator, ID}
→ return at most one proposal deterministically
```

Authority deduplication is inside the evidence path — equivocation never manufactures quorum weight.

The quorum proof chain, end to end

The kernel proves finite counting under equal voting power — then exposes the honest-node obligation.

COMMITTEE $n = 3f + 1$ ›

QUORUM $q = 2f + 1$ ›

INTERSECTION $|Q_1 \cap Q_2| \geq 2q - n$ ›

SUBSTITUTE $\geq f + 1$

At most f Byzantine intersection has at least f + 1 $\Rightarrow \geq 1$ **honest validator**

GLOBAL COUNTING reduces safety to a local invariant: **an honest validator does not double-support.**

Exact Lean coverage — theorem by theorem

Names are part of the audit trail. Green states what is proved; the right edge states what is not.

`quorum_intersection_at_least_f_add_one`	Two 2f+1 quorums inside 3f+1 overlap by ≥ f+1.
`quorum_intersection_contains_honest`	With ≤f Byzantine, the intersection contains an honest validator.
`lemma5_at_most_one_certified_block_per_slot`	Same-slot certificates coincide, given honest single-support.
`conflicting_transactions_cannot_both_gather_quorums`	Conflicting transactions cannot both gather quorums, given honest no-double-vote.

DOES NOT PROVE

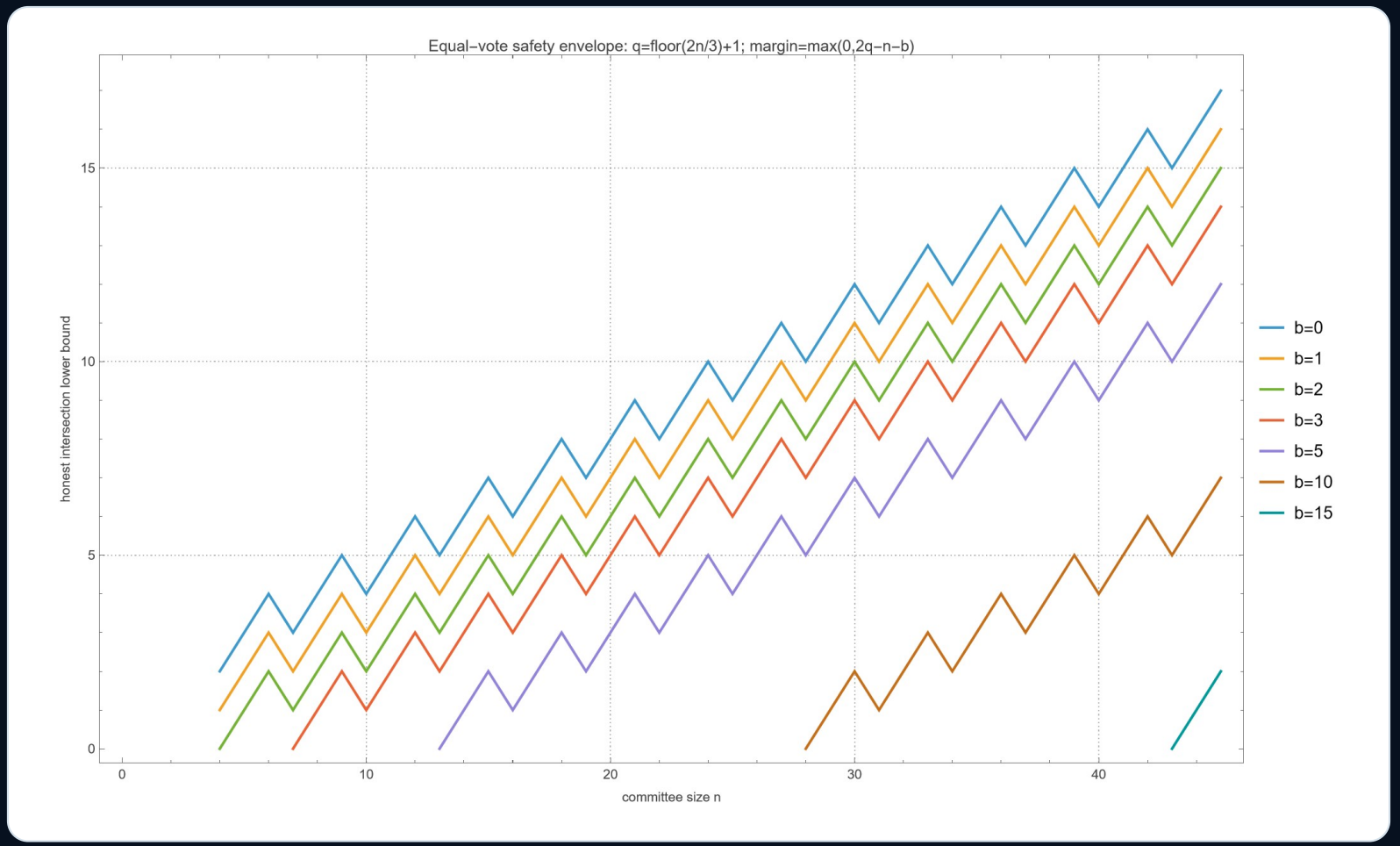
DAG traversal
Direct + indirect decisions
Networking + signatures
Liveness + epoch change
Rust equivalence

PROOF HYGIENE `PASSED — no sorry · admit · axiom · unsafe · native_decide`

LEAN BUILD: 3001 JOBS

Safety envelope: where honest overlap survives

This chart is exact for the equal-authority abstraction; the margin is max(0, 2q − n − b).



READ IT

Positive margin ⇒ at least one honest validator remains in quorum overlap.

BOUNDARY

Zero margin ⇒ this counting argument no longer guarantees honest overlap.

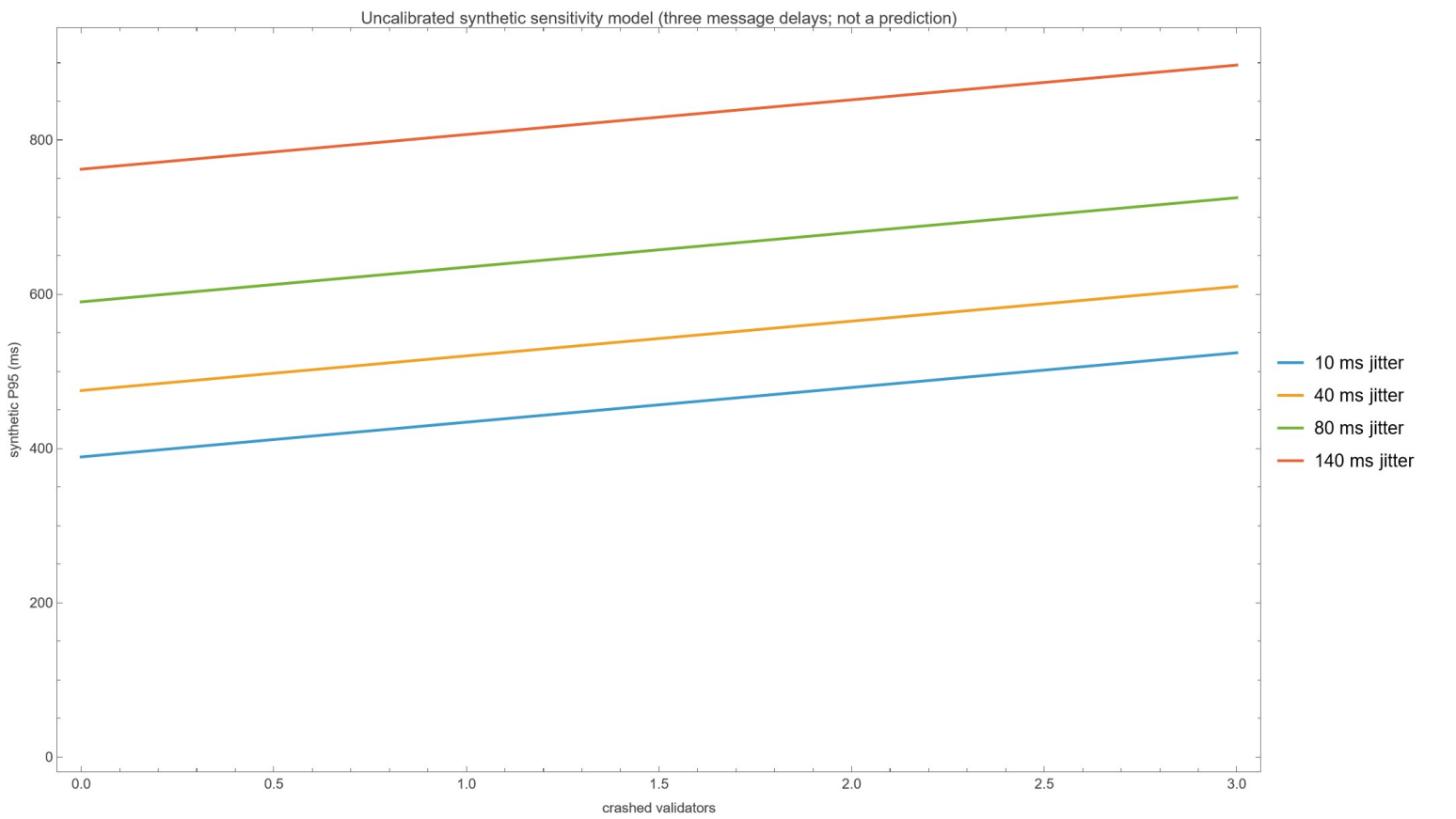
LIMITATION

Production Sui uses stake-weighted voting power. This figure counts equal authorities.

EQUAL AUTHORITY ONLY

Fault sensitivity: useful shape, deliberately uncalibrated

The model asks “how does the response move?” — never “what will production latency be?”



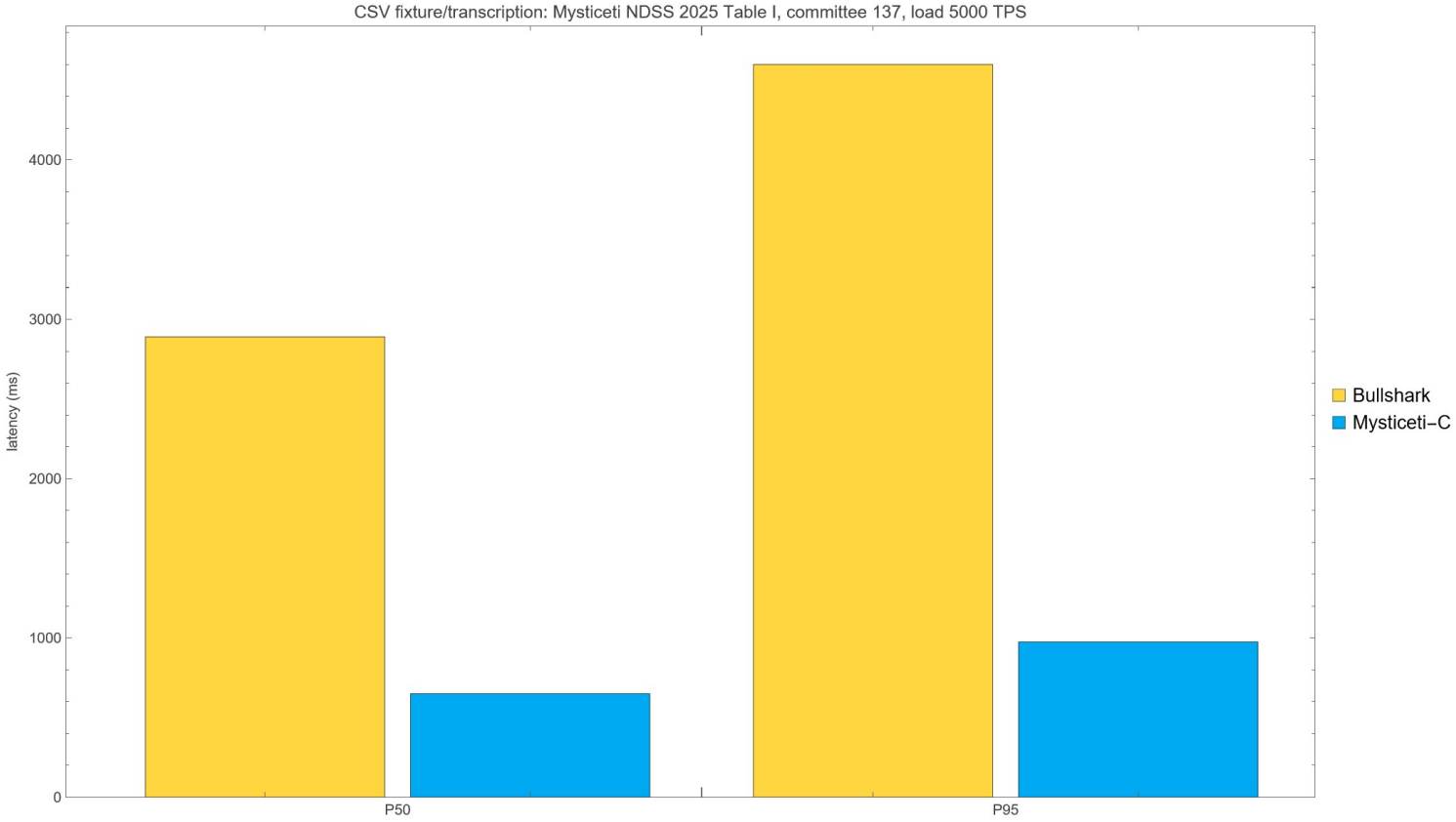
MODEL INPUTS

```
3 × base delay
+ Gaussian jitter
+ 45 ms / crash
+ 25 ms / Byzantine
≥ 250 seeded samples
```

NOT A WAN SIMULATION
NOT A PRODUCTION PREDICTION

Production comparison: faithful transcription, not reproduction

NDSS 2025 Table I · committee 137 · offered load 5,000 TPS. Values are checked against a local CSV fixture.



BULLSHARK

P50 2890 ms
P95 4600 ms

MYSTICETI-C

P50 650 ms
P95 975 ms

CLAIM LIMIT

CSV consistency only. No independent deployment or measurement.

What developers should take into code review

The theorem is small. The implementation obligations it exposes are concrete.

01 COUNT AUTHORITY — Never count blocks or messages as independent voting power.

02 SEPARATE AXES — Crash-performance sensitivity is not Byzantine safety.

03 LOCAL OBLIGATION — Persist “no double-support / no conflicting vote” across recovery.

04 WEIGHTED REALITY — Production Sui uses delegated-stake voting power > 2/3.

05 TRACE ADAPTER PATH — Builder/relay-style timestamp + order traces → provenance → revision-pinned differential tests. No Ethereum builder flow or Sui production implementation claim.

One command. Three lanes. One bounded release gate.

Rust executes and replays; Wolfram conforms and visualizes; Lean checks the mapped theorem.

```
PS> Set-Location "C:\Amp_demos\Mysticeti-Consensus-Digital-Twin"; .\build_all.ps1
```

RUST
26 tests · clippy ×2 OS

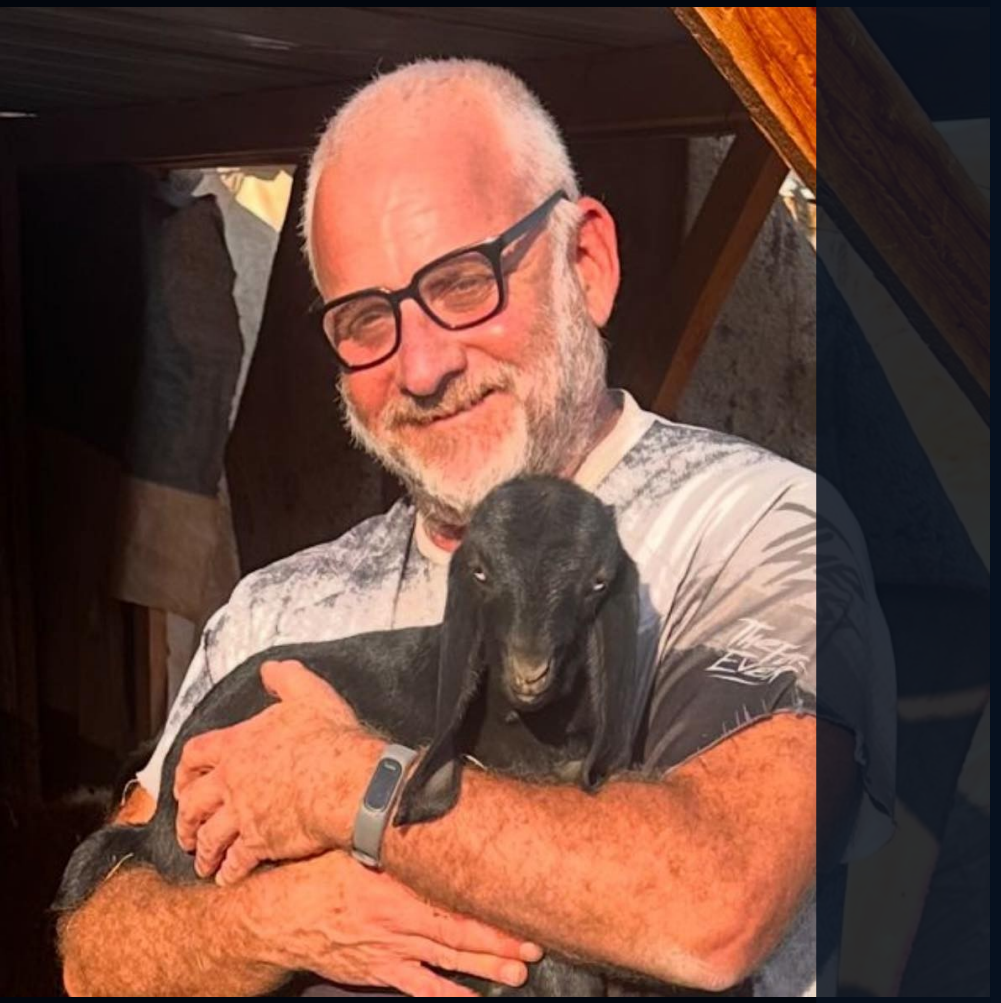
CAMPAIGN
jobs1 = jobs8 hash

LEAN 4
3001 jobs · scan pass

EVIDENCE GATE
audit 29/0 · PASSED

26	5fdc…a6a	12 / 12	29 / 0	PASS
Rust tests	jobs1 = jobs8	Wolfram	audit pass/fail	Lean proof hygiene

COMBINED RELEASE GATE: PASSED



LET'S BUILD THE NEXT BRIDGE

Bring proofs to production conversations.

Rust executes and replays. Wolfram explores and audits. Lean proves bounded mathematics. Amp orchestrates the evidence gate.

Daniel Liezrowice

ESL · Engineering Software Lab

https://eswlab.com/

https://il.linkedin.com/in/liezrowice

https://github.com/zuwasi/mysticeti-proof-to-production-demo

Independent bounded research demo; not affiliated with or endorsed by Mysten Labs or Sui. Lean proves mapped mathematics, not the Rust binary.